

Bitwise Operators in Java

Complete Lecture Notes: Low-level Binary Math, Bit Shifts, Bit Manipulation Tricks & Algorithmic Solutions

Bitwise operators operate directly on the individual bits of integer data types. Because they map directly to machine-level CPU instructions without runtime overhead, bitwise operations are significantly faster than traditional arithmetic operations, making them essential for Competitive Programming and algorithmic optimization.

1. Memory Representation & Binary Fundamentals

In Java, a standard `int` consumes **4 bytes = 32 bits**. Each number is stored in binary representation consisting of zeros and ones.

32-Bit Memory Representation of Integer 5 (`int a = 5`): MSB (Sign Bit) LSB

0 0 0 0 0 | 0 0 0 0 0 0 0 0 | 0 0 0 0 0 1 0 1 | | 0 0 0 0 0 0 0 0 | 0 0 0

3 (Bits 23..16) Byte 2 (Bits 15..8) Byte 1 (Bits 7..0) Byte 4 (Bits 31..24) Byte

Key Terminology:

- **LSB (Least Significant Bit):** The rightmost bit (Bit 0). It determines number parity (even/odd).
- **MSB (Most Significant Bit / Sign Bit):** The leftmost bit (Bit 31).
 - `MSB = 0` → Positive number.
 - `MSB = 1` → Negative number (stored internally in **Two's Complement** format).
- **Set Bit:** Any bit with a value of `1`.
- **Unset / Cleared Bit:** Any bit with a value of `0`.

2. The 6 Core Bitwise Operators in Java

Operator Name	Symbol	Operation Rule	Example (a=5, b=6)	Result
Bitwise AND	<code>&</code>	1 only if both operand bits are 1	<code>0101 & 0110</code>	<code>0100</code> (4)
Bitwise OR	<code> </code>	1 if at least one operand bit is 1	<code>0101 0110</code>	<code>0111</code> (7)
Bitwise XOR	<code>^</code>	1 if operand bits are different; 0 if same	<code>0101 ^ 0110</code>	<code>0011</code> (3)
Bitwise NOT (Complement)	<code>~</code>	Inverts all bits (0 → 1, 1 → 0)	<code>~5</code>	-6
Left Shift	<code><<</code>	Shifts bits left, fills right with 0s ($a \times 2^k$)	<code>5 << 1</code>	10
Signed Right Shift	<code>>></code>	Shifts bits right, preserves MSB sign ($a / 2^k$)	<code>5 >> 1</code>	2
Unsigned Right Shift	<code>>>></code>	Shifts bits right, always fills left with 0s	<code>-5 >>> 1</code>	2147483645

3. Deep Dive: Bitwise NOT (~) & Two's Complement

Why does `~5` produce `-6` in Java? Understanding this requires analyzing 1's Complement and 2's Complement representation.

Step-by-Step Execution of `~5`: 1. Binary of 5 (32-bit): 00000000 00000000 00000000 00000101 2. Bitwise NOT (`~5`) flips every single bit: 11111111 11111111 11111111 1111010 3. The MSB is 1 -> The JVM recognizes this as a Negative Integer. 4. Negative numbers are interpreted by taking their Two's Complement: a. Take 1's Complement (Flip bits): 00000000 00000000 00000000 00000101 (5) b. Add 1 (2's Complement): 00000000 00000000 00000000 00000110 (6) 5. Adding the negative sign yields: -6

💡 General Formula for Bitwise NOT

For any integer N in Java: $\sim N = -(N + 1)$

- $\sim 5 = -(5 + 1) = -6$
- $\sim 0 = -(0 + 1) = -1$
- $\sim(-10) = -(-10 + 1) = 9$

4. Left Shift (<<) and Right Shift (>>) Mechanics

Left Shift ($a \ll k$) — Fast Multiplication

Shifts bits left by k positions. Fills emptied right positions with 0 .

Mathematical Equivalent: $a \ll k = a \times 2^k$

```
int a = 5;
System.out.println(a << 1); // 5 * 2^1 = 10
System.out.println(a << 2); // 5 * 2^2 = 20
System.out.println(3 << 2); // 3 * 2^2 = 12
```

Extreme Case: Shifting a set bit into bit 31 turns the number negative; shifting by ≥ 32 bits wraps modulo 32 in Java.

Right Shift ($a \gg k$) — Fast Division

Shifts bits right by k positions. Discards overflowing LSB bits.

Mathematical Equivalent: $a \gg k = a / 2^k$

```
int n = 100;
System.out.println(n >> 1); // 100 / 2^1 = 50
System.out.println(n >> 2); // 100 / 2^2 = 25
System.out.println(16 >> 2); // 16 / 4 = 4
```

Note: $\gg$ preserves the sign bit (arithmetic shift), whereas $\gg>$ fills with zero (logical shift).

5. Crucial XOR (^) Properties

XOR evaluates to 0 for identical bits and 1 for distinct bits. This yields three powerful mathematical properties:

1. **Self-Cancellation:** $A \wedge A = 0$ (Any number XORed with itself cancels out).
2. **Identity Element:** $A \wedge 0 = A$ (Any number XORed with zero remains unchanged).
3. **Commutative & Associative:** $A \wedge B \wedge A = (A \wedge A) \wedge B = 0 \wedge B = B$.

6. Standard Problem Patterns & Code Implementations

Pattern 1: Fast Even/Odd Check (Replacing Modulo)

Intuition: Even numbers in binary always have an LSB of 0 ; odd numbers always have an LSB of 1 . Performing $n \& 1$ isolates the LSB.

```
public static boolean isEven(int n) {
    return (n & 1) == 0; // Faster than (n % 2 == 0)
}
```

Pattern 2: Swap Two Numbers Without a Temporary Variable (Using XOR)

```
int a = 5, b = 6;
a = a ^ b; // a contains (5 ^ 6)
b = a ^ b; // b = (5 ^ 6) ^ 6 = 5 ^ (6 ^ 6) = 5 ^ 0 = 5
a = a ^ b; // a = (5 ^ 6) ^ 5 = (5 ^ 5) ^ 6 = 0 ^ 6 = 6
// Result: a = 6, b = 5
```

Pattern 3: Find Unique Element (All Other Elements Appear Twice)

Intuition: XORing all array elements cancels out all paired duplicates ($x \wedge x = 0$), leaving only the unique element.

```
public static int findSingleElement(int[] arr) {
    int unique = 0;
    for (int num : arr) {
        unique ^= num;
    }
    return unique;
}
```

Pattern 4: Count Set Bits

Method 1: Shift & Check LSB — $O(\text{ext}\{\text{Bits}\})$

```
public static int countSetBits(int n) {
    int count = 0;
    while (n > 0) {
        if ((n & 1) != 0) count++;
        n = n >> 1;
    }
    return count;
}
```

Method 2: Brian Kernighan's Algorithm — $O(\text{ext}\{\text{Set Bits}\})$

```
public static int countSetBitsFast(int n) {
    int count = 0;
    while (n > 0) {
        n = n & (n - 1); // Clears lowest set bit
        count++;
    }
    return count;
}
```

Pattern 5: Check If a Number Is a Power of Two

Core Mathematical Insight: A power of two has exactly **one set bit** (e.g., $16 = 10000_2$). The number immediately preceding it has all lower bits set ($15 = 01111_2$). Thus, $n \ \& \ (n - 1)$ evaluates to 0 .

```
public static boolean isPowerOfTwo(int n) {
    if (n <= 0) return false;
    return (n & (n - 1)) == 0;
}
```

Pattern 6: Remove the Rightmost (Lowest) Set Bit

$n \ \& \ (n - 1)$ removes the rightmost set bit from n .

Example: $N = 10$ (1010). $N - 1 = 9$ (1001). $10 \ \& \ 9 = 1000 = 8$.

Pattern 7: Isolate / Get the Rightmost (Lowest) Set Bit

$n \ \& \ (-n)$ isolates the lowest set bit and clears all other bits.

Why it works: $-n$ is stored in 2's complement format ($\sim n + 1$). The lowest set bit remains 1 in both, while all other bits differ.

```
int n = 10; // 1010_2
int lowestBit = n & (-n); // Result: 0010_2 = 2
```

7. Master Formula Cheat Sheet

Operation / Query	Bitwise Expression	Mechanism & Explanation
Check Even / Odd	$(n \ \& \ 1) == 0$	Checks if LSB is 0 (Even) or 1 (Odd).
Multiply by 2^k	$n \ \<< \ k$	Fast left shift multiplication.
Divide by 2^k	$n \ \>> \ k$	Fast right shift integer division.
Check Power of 2	$n > 0 \ \&\& \ (n \ \& \ (n - 1)) == 0$	Power of 2 contains exactly one set bit.
Clear Lowest Set Bit	$n = n \ \& \ (n - 1)$	Turns the rightmost 1-bit into 0.
Extract Lowest Set Bit	$n \ \& \ (-n)$	Keeps only the lowest 1-bit active.
Bit Inversion (NOT)	$\sim n = -(n + 1)$	Flips all 32 bits.
Swap Variables	$a \ \wedge = \ b; \ b \ \wedge = \ a; \ a \ \wedge = \ b;$	In-place swap without extra space.

⚠ Common Beginner Traps

- **Operator Precedence Trap:** Bitwise operators ($\&$, $|$, $\wedge$) have *lower* precedence than comparison operators ($==$, $!=$). Always wrap expressions in parentheses: `if ((n & 1) == 0)` instead of `if (n & 1 == 0)` (which causes compile errors!).
- **Negative Numbers & Power of 2:** $n \ \& \ (n - 1) == 0$ returns `0` for `0` and negative boundaries. Always check `n > 0` first.



Practice & Homework Challenges

1. Implement fast exponentiation (a^b) in $O(\log b)$ using bitwise shifts.
2. Find the missing number in an array of numbers from 1 to N using XOR.
3. Find two non-repeating unique elements in an array where every other element appears twice.
4. Write a function to convert a decimal integer to binary format string manually using bit extraction.